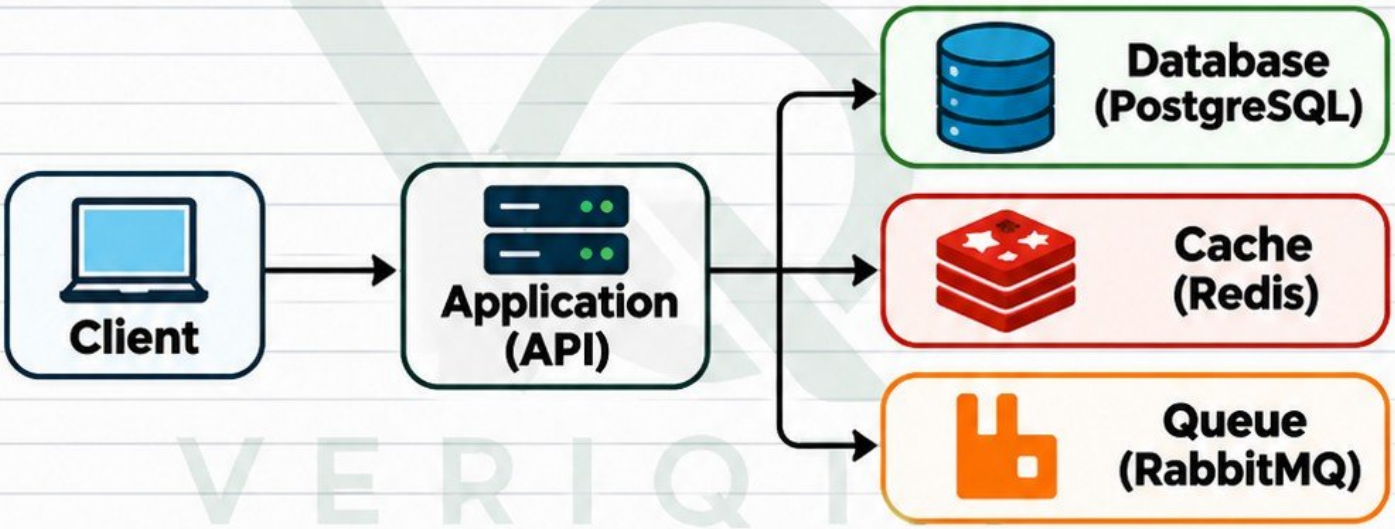
VERIQTA

From One Server to a Real System:

Add the Database, Cache and Queue



Architecture & Design | Performance & Scaling | Reliability & Resilience | Troubleshooting & Incidents | Production Practices

A Practical Engineering Guide to Evolve a Single Server into a Scalable, Reliable and Production-Ready System

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

PAGE 01

The Single Server That Became a Business Risk

The Scenario

A working application runs on one Linux server. It handles customer orders, stores data locally, runs background tasks, and is deployed directly on the same machine.

Current Setup: One Server



Single Linux Server (Everything Together)



Application (API + Web)



Database (Local Files)



Background Tasks (Emails, Reports, Sync)



Local Storage (Uploads, Logs, Exports)



Deployment (Direct on Server)

Order Request Flow (Today)

1. Client sends an order to the API.
2. App validates and writes to the local database.
3. Background task sends email and updates reports.
4. Files (invoices, logs) are saved on the same disk.

Shared Failure Domains

- One machine for compute, data, tasks, and storage.
- Single disk and resources (CPU, memory, I/O).
- Deployment and app runtime on the same host.
- A failure can take everything down at once.

What Can Go Wrong?

- Downtime: app crash, OS issue, or failed deployment.
- Data loss: disk corruption or accidental file deletion.
- Duplicate operations: retries on timeouts with no queue.
- Performance collapse: heavy tasks compete with orders.

Senior Decision

- Separate components with different failure, scaling, or data safety needs (e.g., database, queue, cache).
- Keep components that can safely co-exist, but isolate their resources (e.g., background workers, logs).

Establish the Evidence Before the Architecture

The Goal

Understand the current system with real data before adding a database, cache, and queue. Measure, trace, and find the actual bottleneck.

Key Evidence to Capture

- **Latency:** p95 and p99 (API and key operations)
- **Throughput:** requests/second (RPS) and trends
- **Worker Utilization:** busy time, queue depth, and wait time
- **Disk I/O:** read/write rate, I/O wait, and saturation
- **Memory Pressure:** usage, cache, and swap activity
- **Database Wait Time:** locks, slow queries, and I/O
- **Failed Business Transactions:** errors, timeouts, and retries

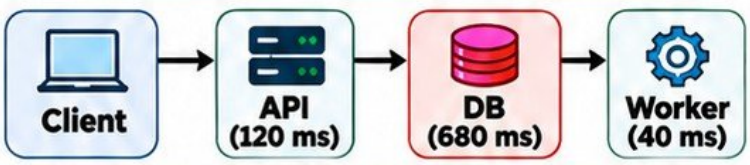
How to Investigate

1. Trace an order request end-to-end (using request tracing).
2. Inspect the server with Linux tools (top, vmstat, iostat, pidstat).
3. Analyze database behavior (locks, slow queries, wait events).
4. Correlate metrics and logs to find the real bottleneck.

Bottleneck Types to Identify

- **Compute-bound:** CPU saturation, high load, long processing time.
- **Storage-bound:** high disk I/O, I/O wait, or slow data access.
- **Locking-bound:** database locks, high wait time, and blocked transactions.
- **Dependency-bound:** external API latency, timeouts, or high error rate.

Example: Order Request Trace



Total: 840 ms | p95: 920 ms | p99: 1.3 s

Production Insight

Adding a database, cache, and queue can make a poorly understood bottleneck harder to diagnose.

VERIQTA

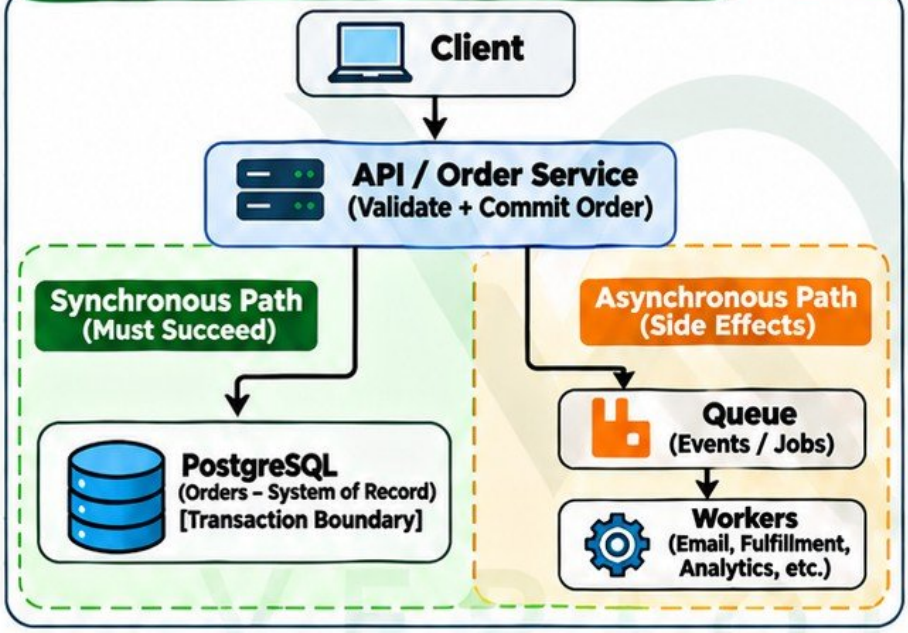
PAGE 03

Draw the Failure Boundaries Before Drawing the Architecture

The Idea

Design clear request and data flows, define what must be synchroıous (order commit) and what can be asynchronous (side effects), and set ownership, transaction boundaries, and failure domains.

Target Flow and Failure Domains



Define the Boundaries

- **Ownership:** PostgreSQL owns the order state (single source of truth).
- **Transaction Boundary:** Order validation, write, and commit are atomic.
- **Network Trust Zones:** Public (client) → App (private) → Data/Queue (private).
- **Always Available:** API and database must be available to accept orders.

Failure Boundary Examples

Failure	Blast Radius	Business Impact
Queue down	Async (side effecs) only	Order accepted, notifications delayed
Worker crash	Background jobs	No email/fulfillment, order is safe
Database down	Order commit path	Cannot accept new orders (high impact)
Cache down	Read performance	Slower reads, system still works

Architecture Trade-Off

- More services can improve scalability and isolation, but also increase operational complexity, new failure modes, and debugging time.
- Separate only what creates real resilience, scalability, or business value.

Production Insight: A good architecture starts with clear failure boundaries and ownership, not with more components.

VERIQTA

PAGE 04

Extract PostgreSQL Without Creating a Data-Loss Window

The Goal

Move from local persistence to a dedicated PostgreSQL without losing or duplicating orders, while keeping the system safe and reversible.

Migration Approaches

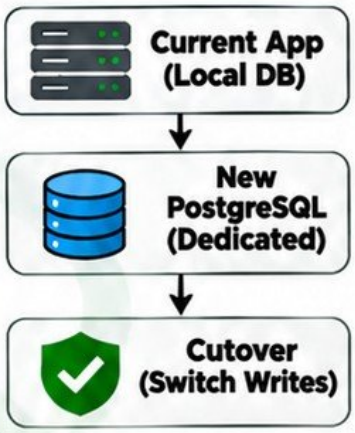
Option 1: Controlled Downtime

- Stop writes, copy data, switch app, validate, then resume.
- Pros: simple and predictable.
- Cons: service downtime (minutes–hours).

Option 2: Snapshot + CDC (Minimal Downtime)

- Take a snapshot, then sync ongoing changes (CDC) until cutover.
- Pros: near-zero downtime, keeps service live.
- Cons: more complex setup and monitoring.

Migration Flow (CDC-Based)



1. Take initial data snapshot.
2. Stream changes (CDC) and keep in sync.
3. Fence old writes, switch app, and verify.

Critical Checks Before Cutover

- **Write Fencing:** block or redirect writes to the old database.
- **Primary Keys:** preserve IDs and sequences (e.g., orders, users).
- **Schema Compatibility:** same version, constraints, and indexes.
- **Data Reconciliation:** compare row counts, checksums, and sample records.
- **Rollback Plan:** documented steps to return to the old database if needed.

Key Risks and Mitigations

Risk	Mitigation
Missing orders during migration	Use CDC, validate counts, and reconcile.
Duplicate orders	Enforce unique constraints and idempotent writes.
Schema mismatch	Run schema diff and migration scripts first.
Partial cutover failure	Keep rollback ready and test in a staging clone.

Production Failure: A successful database import is not proof that the cutover preserved every committed order.

VERIQTA

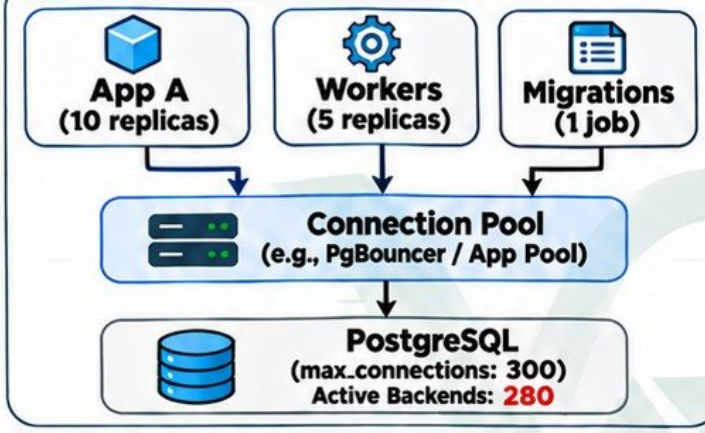
PAGE 05

Connection Pools Can Take Down a Healthy Database

The Problem

The database is healthy, but the application overwhelms it with too many or poorly managed connections, long transactions, and scaling spikes.

How It Works (Flow)



Failure Modes to Watch

- **Pool Exhaustion:** All pool connections are in use; new requests wait or fail.
- **Idle-in-Transaction:** Connections sit open, holding locks and resources.
- **Lock Contention:** Long transactions block others, increasing latency.
- **Connection Storm:** App scaling or restarts create a sudden surge of connections.

Key Diagnostics (PostgreSQL)

```
# Active connections by state
SELECT state, count(*) FROM pg_stat_activity
GROUP BY state;

# Long running transactions
SELECT pid, state, now()-xact_start AS duration,
query FROM pg_stat_activity
WHERE state = 'active' ORDER BY duration DESC;

# Lock contention
SELECT FROM pg_locks pll
JOIN pg_stat_activity ps ON pl.pid = ps_pid
WHERE NOT granted;
```

Connection Budget Example

Component	Replicas	Pool/Replica	Total
Web API	10	20	200
Workers	5	10	50
Migrations	1	5	5
Admin / Ops	–	10	10
Total (≤ max_connections)			265 / 300

PgBouncer: When and Why

- **Use Transaction Pooling:** for stateless, short transactions (most web apps).
- **Use Session Pooling:** only if you need session features (e.g., temp tables, LISTEN/NOTIFY, prepared statements).

Senior Decision

- Budget connections across all clients (apps, workers, migrations, ops).
- Keep transactions short and predictable.
- Use PgBouncer to control concurrency.
- Set alerts on pool usage, long transactions, and active backends.

Production Insight: A database can stay healthy and still go down if you let connections, transactions, and locks run unchecked.

VERIQTA

PAGE 06

The Query That Passes Testing but Collapses Under Load

The Scenario

A query is fast in testing (~50 ms) but becomes slow in production (> 3 s) as real data and traffic grow. You need to find out why and fix it with the right trade-off.

The Problem Query (Example)

```
SELECT o.id, o.total, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.created_at >= NOW() - INTERVAL '7 days'
ORDER BY o.created_at. DESC
LIMIT 100;
```

EXPLAIN (ANALYZE, BUFFERS) Evidence

```
Sort (actual time=2800.1 ms rows=100)
 -> Nested Loop (actual time= 0.3...2750 ms)
      Seq Scan on orders
       (actual time= 0. 2...2400 ms rows=5,000,000)
       Filter: created_at >= ....
      Buffers: shared hit=34000 read=12000
```

Likely Root Causes

- **Low Selectivity:** filter matches a large portion of the table.
- **Skewed Data:** most orders fall within the last 7 days.
- **N+1 Pattern:** multiple repeated queries inside the application.
- **Lock Contention:** concurrent writes or long-running transactions.
- **Missing / Wrong Index:** forces a sequential scan and heavy sorting.

Investigation Workflow

1. Check top queries:
   `SELECT * FROM pg_stat_statements ORDER BY total_time DESC LIMIT 5;`
2. Inspect the plan:
   `EXPLAIN (ANALYZE, BUFFERS) <query>;`
3. Check lock waits:
   `SELECT * FROM pg_stat_activity WHERE wait_event_type = 'Lock';`
4. Validate data & indexes:
   `SELECT reltuples::bigint AS rows FROM pg_class WHERE relname = 'orders';`

A Better Plan: Index + Query Tuning

```
CREATE INDEX idx_orders_created_at
ON orders (created_at DESC);
```

Metric	Before	After
Execution Time (p95)	2.8 s	48 ms
Rows Scanned	5,000,000	100
Buffers Read	12,000	120

The Trade-Off

✓ Benefits

- Faster reads and lower latency.
- Better user experience under load.

✗ Costs

- Higher write cost (write amplification).
- More storage space.
- Index maintenance (vacuum, rebuild, and bloat).

Senior Decision

- Add the right index based on query pattern and data distribution.
- Remove N+1 patterns (use joins or batching).
- Keep transactions short and avoid unnecessary locks.
- Validate with real production-like data and traffic before rollout.

Performance Trade-Off: An index can accelerate reads while increasing write amplification, storage consumption, and maintenance.

VERIQTA

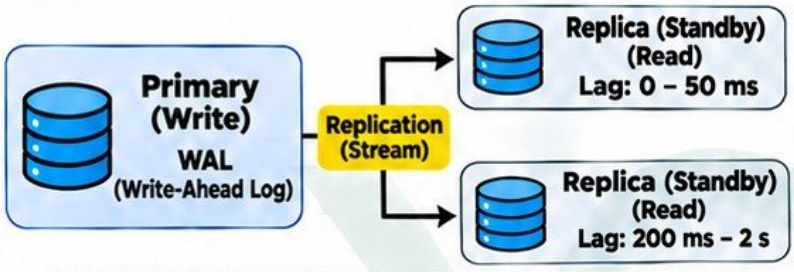
PAGE 07

Database Failover Is Not the Same as Data Safety

The Point

Failover can bring your app back online quickly, but it does not automatically guarantee that every committed transaction is preserved.

PostgreSQL Replication and Failover



WAL: a sequential log of all changes, streamed to replicas.

Replication Modes

Asynchronous (default): High performance, lower latency, but can lose recent transactions during failover.

Synchronous: Confirms a write on the primary only after a replica receives it. Safer, with higher write latency.

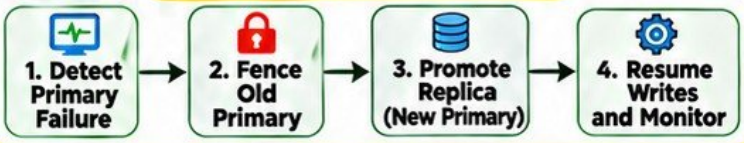
Key Concepts to Understand

- **Write-Ahead Logging (WAL):** Records every change for recovery and replication.
- **Replication Lag:** The delay between the primary and replica.
- **Failover Fencing:** Ensure the old primary cannot keep accepting writes (split-brain).
- **Split-Brain:** Two active primaries writing different data (causes data loss).
- **Recovery Timeline:** Time to restore service (RTO) and data (RPO).

RPO and RTO: Define for Your Business

Metric	Meaning	Example
RPO (Recovery Point Objective)	Maximum acceptable data loss.	≤ 5 seconds (e-commerce orders)
RTO (Recovery Time Objective)	Maximum time to restore service.	≤ 2 minutes (orders, payments, checkout)

Failover Flow (Safe)



Common Failure Scenarios

Scenario	Result
Failover with replication lag	Service restores but recent orders are lost.
No fencing (split-brain)	Two primaries, data divergence, manual recovery.
Unrealistic RPO/RTO	Business impact (lost sales, corrupt state, long downtime).

Senior Decision Checklist

- Choose replication mode (async vs sync) per business risk.
- Set realistic RPO and RTO with stakeholders.
- Implement failover fencing (e.g., Patroni, Stolon, or kube).
- Monitor replication lag and WAL apply rate.
- Test failover and recovery regularly (not just in theory).

Incident Lesson: A promoted replica can restore service while silently omitting transactions that the former primary acknowledged.

VERIQTA

PAGE 08

Decide What Must Never Be Cached

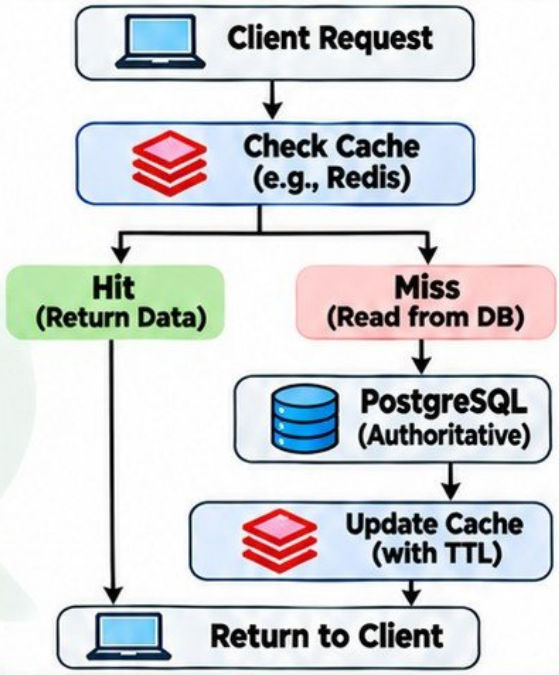
The Goal

Be deliberate about what to cache, what not to cache, and when stale data is acceptable, while keeping the database as the single source of truth for orders and inventory.

Data Classification by Consistency

Data / Example	Consistency Need	Cache?	Reason
Order Records (e.g., order, payment)	Strong (Always correct)	No	Drives business state and money.
Product Descriptions (name, image, specs)	Eventual (Can be stale)	Yes	Changes rarely; low business risk.
Inventory Availability (stock count)	Strong (Authoritative)	No	Must be accurate for inventory deduction.
Account Balances (credit, wallet)	Strong (Always correct)	No	Financial data must be real-time.
Derived Read Models (e.g., top products, analytics)	Eventual (Approximate OK)	Yes	Aggregated data tolerates delay (e.g., minutes).

Cache-Aside Pattern (Read)



Rules to Follow

- **Authoritative Source:** PostgreSQL (or your primary DB) controls orders and inventory.
- **Use Cache-Aside:** read from cache, on miss read from DB and populate with a TTL.
- **Set the Right TTL:** short TTL for fast-changing data (e.g., product price), longer TTL for static data.
- **Invalidate on Change:** clear or update cache after critical writes (product updates, inventory adjustments).

Trade-Offs and Risks

Benefit	Trade-Off / Risk
Faster reads and lower DB load.	Can serve stale data (if not invalidated).
Better user experience (lower latency).	Extra complexity (TTL, invalidation, consistency).
Handles high read traffic cost-effectively.	Not suitable for data that must be exact (orders, inventory, balances).

Senior Decision: Do not use an eventually consistent cached stock count to authorize an inventory deduction.

VERIQTA

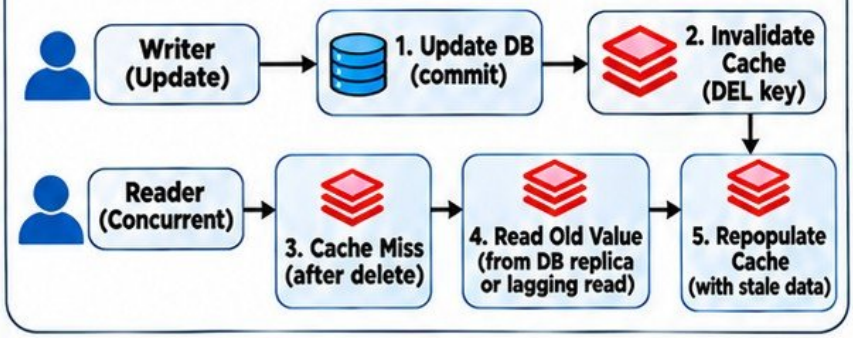
PAGE 09

The Cache Invalidation Race Nobody Notices in Staging

The Problem

After an update, the cache is invalidated, but concurrent requests can repopulate the old value, causing customers to see stale data in production.

The Race Condition (How It Happens)



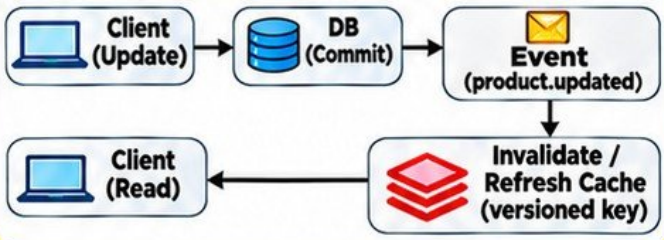
Key Techniques to Prevent It

- **Versioned Keys:** use a version (e.g., product:123:v5) and bump on every update.
- **Event-Driven Invalidation:** publish an update event (e.g., product.updated) and invalidate/refresh consistently.
- **TTL Boundaries:** set a sensible TTL to limit the lifetime of stale data.
- **Stale-Write Prevention:** use conditional writes (e.g., SET with version/ETag) to avoid overwriting newer data.

Sequence Timeline (Real Example)

Step	Event	Result
1	Update product price to $49	DB commit succeeds
2	Invalidate cache (DEL key)	Cache is empty
3	A concurrent read happens	Cache miss
4	Reader fetches old price (e.g., from replica or lagging read)	Old value: $39
5	Reader repopulates cache with $39	Stale value serves to customers

Better Design (Safer Flow)



Common Pitfalls

- Using DEL without handling concurrent reads.
- No versioning, so old data can overwrite new data.
- Replicating reads from a lagging database.
- No TTL, so stale data stays too long.

Senior Decision Checklist

- Use versioned keys for mutable data.
- Make invalidation event-driven (reliable and ordered).
- Set appropriate TTL for fallback safety.
- Prevent stale writes with conditional updates (version/ETag).
- Test concurrent update + read scenarios under load.

Production Failure: A database transaction can commit successfully, yet a delayed cache fill can restore the previous value and serve it to customers.

VERIQTA

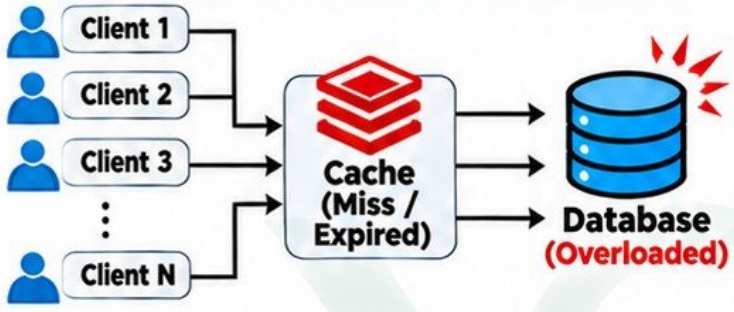
PAGE 10

When a Cache Miss Becomes a Distributed Outage

The Situation

A surge of cache misses can send a flood of requests to the database, overwhelming it and causing a wider system outage across services.

Cache Stampede (What Happens)



Common Triggers

- **Cache Stampede:** many requests rebuild the same key at once.
- **Hot Keys:** a few keys receive disproportionate traffic.
- **Synchronized Expiry:** large number of keys expire at the same time.
- **Eviction Pressure:** useful keys are removed due to memory limits.
- **Connection Exhaustion:** too many Redis connections (or a connection leak).

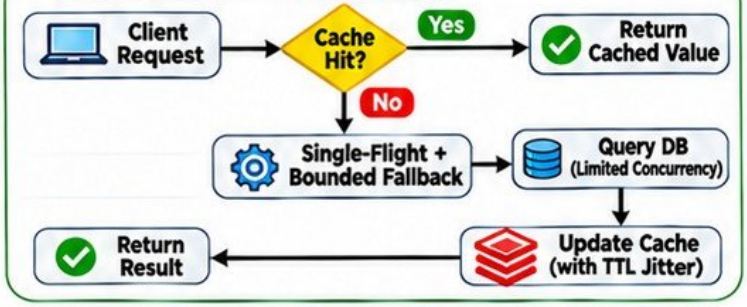
Key Mitigation Strategies

- **Single-Flight (Request Coalescing):** allow only one request to rebuild a key; others wait for the result.
- **TTL Jitter:** add random variation to TTL (e.g., 300 s ± 20%) to avoid synchronized expirations.
- **Stale-While-Revalidate:** serve a slightly stale value while refreshing the cache in the background.
- **Bounded DB Fallback:** limit the number of concurrent DB queries (e.g., via a semaphore or rate limiter) when the cache is unavailable.

Design Options Comparison

Strategy	Benefit	Trade-Off
Single-Flight	Prevents request storms.	Adds coordination complexity.
TTL Jitter	Spreads cache expirations.	Slightly less predictable expiry.
Stale-While-Revalidate	Keeps service fast and available.	Users may see briefly stale data.
Bounded Fallback	Protects the database.	May return errors under heavy load if limits are strict.

A Safer Read Flow



Senior Decision Checklist

- Identify hot keys and high-risk endpoints.
- Use single-flight for expensive keys.
- Add TTL jitter to avoid mass expirations.
- Use stale-while-revalidate for read-heavy data.
- Apply a bounded DB fallback (rate limit / concurrency).
- Monitor cache hit rate, Redis connections, and DB load.

Reliability Insight: Cache failure must not trigger unlimited database queries that overwhelm the authoritative system.

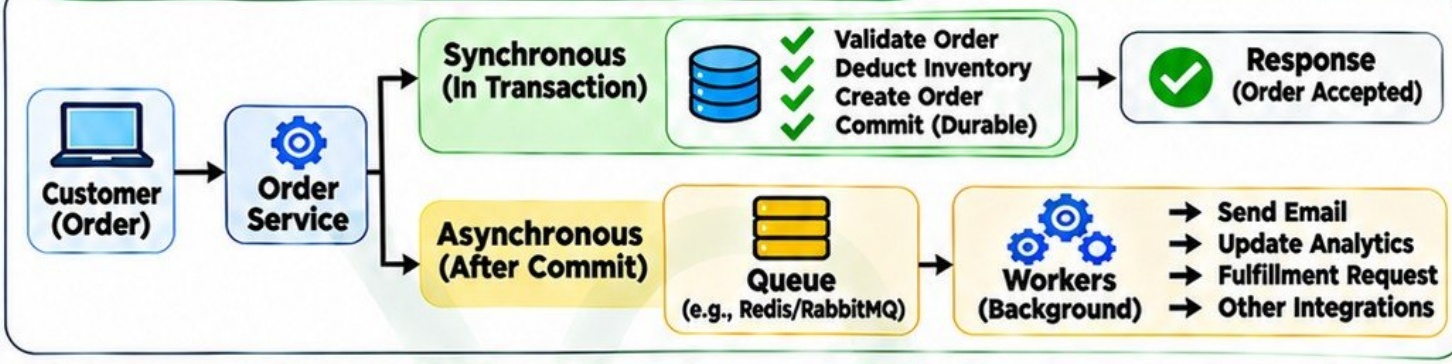
VERIQTA

Separate the Customer Response From the Work That Follows

The Principle

Give the customer a fast, reliable response after you secure the order, and move non-critical work to asynchronous processes.

Order Flow: Synchronous vs Asynchronous



What Stays in the Order Transaction?

- Order validation (customer, item, price).
- Inventory reservation/deduction.
- Order record creation (authoritative).
- Idempotency check (prevent duplicates).
- Single, atomic commit (durable acceptance).

What Runs Asynchronously?

- Email / SMS notifications (order confirmed).
- Analytics and reporting updates.
- External fulfillment / shipping requests.
- Non-critical integrations (e.g., CRM, loyalty).
- Long-running tasks (e.g., invoice generation).

Worker Design and Reliability

Aspect	Recommendation
Durable Acceptance	Persist the order, then publish a message to a durable queue.
Worker Responsibility	Own a single, clear task (e.g., send email, call fulfillment API).
Retry Policy	Use exponential backoff with a max retry limit (e.g., 3–5 attempts).
Failure Ownership	The worker owns failure handling (log, retry, and move to a dead-letter queue).

Trade-Offs: Queues and Async Processing

Benefits

- Faster customer response (lower latency).
- Reduces synchronous coupling between services.
- Handles spikes with buffering and retries.

Trade-Offs

- Introduces delivery delays (not immediate).
- Messages can be replayed (need idempotent handlers).
- Adds operational state (queues, retries, DLQ, monitoring).

Architecture Trade-Off: Queues reduce synchronous coupling but introduce delivery delays, replay, and additional operational state.

VERIQTA

PAGE 12

The Dual-Write Failure Between PostgreSQL and the Queue

The Problem

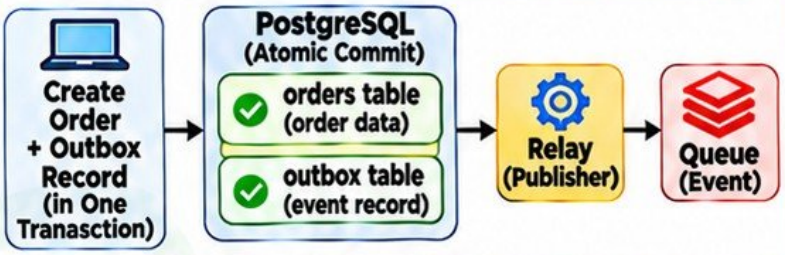
There is a gap between committing the order in PostgreSQL and publishing its event to the queue, which can leave the system in an inconsistent state if a failure occurs in between.

The Failure (Without an Outbox)



Result: The order exists in PostgreSQL, but no fulfillment event is published.

The Solution: Transactional Outbox



The order and its outbox record commit together, and a separate relay reliably publishes the event to the queue.

Transactional Outbox Flow

1. Begin transaction.
2. Insert order into orders table.
3. Insert event into outbox table (e.g., order.created with event_id).
4. Commit both records atomically.
5. Relay reads outbox and publishes to the queue, with retries and backoff.

Key Implementation Details

Aspect	Approach
Outbox Table	Store event_id, event_type, payload, status, retry_count, next_retry_at, created_at.
Relay (Publisher)	Background worker that reads pending outbox records and publishes to the queue.
Retries	Use exponential backoff and max retry policy (e.g., 3, 5, 10 ... attempts).
Event Identifier	Use a unique, idempotent event_id (e.g., UUID) to prevent duplicate processing.
Idempotency	Consumers handle duplicate events safely (process once, ignore repeats).

Benefits

- Eliminates the dual-write gap.
- Ensures the event is eventually published.
- Improves reliability and auditability.
- Supports safe retries and idempotency.

Trade-Offs

- Adds an outbox table and a relay process to operate.
- Slightly increases write and storage overhead.
- Introduces eventual (not immediate) event delivery.
- Requires monitoring (lag, retries, and stuck events).

Production Failure: The order exists in PostgreSQL, but the process crashes before publishing the fulfillment event.

VERIQTA

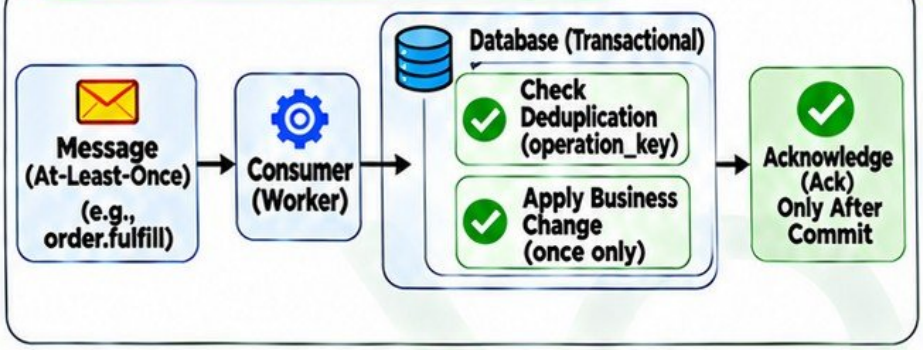
PAGE 13

Exactly-Once Business Effects Without Exactly-Once Delivery

The Reality

Message delivery from a queue is typically at-least-once (duplicates can happen), so you must make the consumer idempotent to ensure the business effect happens only once.

Flow: Idempotent Consumer



Key Design Elements

- **Unique Business Key:** use a stable idempotency key (e.g., order_id, payment_id).
- **Durable Deduplication:** store processed keys with status and timestamp.
- **Transactional Update:** apply the business change and record the key in one transaction.
- **Safe Acknowledgment:** ack the message only after a successful commit.

Failure and Redelivery Scenarios

Scenario	What Happens	How It's Handled
Crash before ack (after commit)	Message is redelivered.	Deduplication finds the key and skips the change.
Crash before commit	No business change was made.	Message is retried and processed normally.
Duplicate delivery (e.g., network)	Same message arrives again.	Idempotency key prevents a second business effect.

Idempotency Data Model (Example)

Column	Type	Purpose
operation_key	UUID (PK)	Unique business operation (e.g., order_id)
status	TEXT	processing | completed | failed
result_ref	TEXT	Reference to the outcome (e.g., fulfillment_id)
created_at	TIMESTAMP	When the operation was received
updated_at	TIMESTAMP	Last status change

Applying Idempotency to External APIs



Best Practices Checklist

- Use a meaningful, unique business-operation key.
- Persist deduplication records durably.
- Wrap state change and key recording in one transaction.
- Acknowledge only after a successful commit.
- Pass an idempotency key to external APIs (payments, shipping).

Trade-Offs

Benefit	Trade-Off
Prevents duplicate business effects.	Requires additional storage (deduplication records).
Works with at-least-once delivery.	Adds implementation complexity.
More resilient to failures and retries.	Needs support from external APIs (idempotency keys).

Key Takeaway

Deduplicate the business effect, not merely the message transport. External APIs must also support idempotency.

Senior Decision: Use at-least-once delivery with idempotent consumers and external API idempotency to achieve exactly-once business effects.

VERIQTA

Poison Messages, Retry Storms and the Dead-Letter Trap

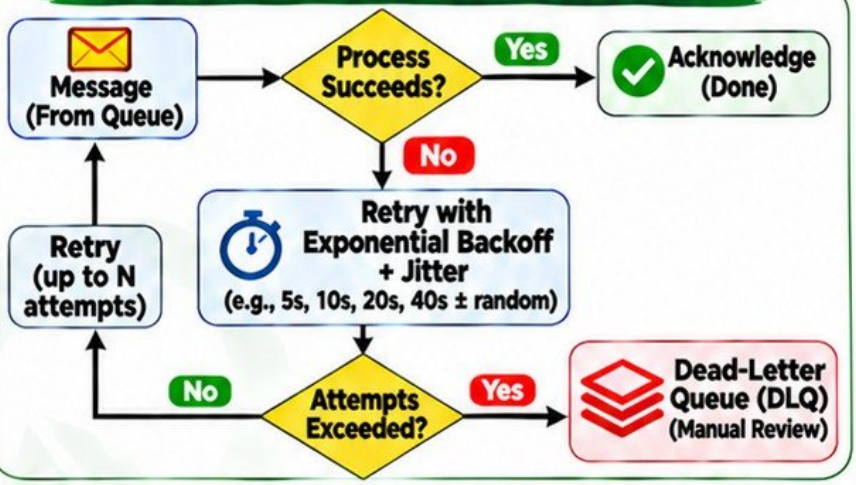
The Problem

Failed messages can keep retrying, overload your system, and still end up in a dead-letter queue (DLQ) without being actually resolved.

Types of Failures

- **Transient Error** (Retryable): temporary issue (e.g., network timeout, DB contention).
- **Permanent Failure (Non-Retryable):** invalid data (e.g., missing fields, violates business rules).
- **Ambiguous Outcome:** unknown result (e.g., external API timeout — the operation may have succeeded).

Retry Flow with Backoff and DLQ



Retry Strategy (Example)

Attempt	Base Delay	Jitter (±)	Next Wait (Example)
1	5s	±2s	3s – 7s
2	10s	±3s	7s – 13s
3	20s	±5s	15s – 25s
4	40s	±10s	30s – 50s
5	80s	±20s	60s – 100s

Dead-Letter Handling

- Send failed messages to a DLQ after N attempts.
- Capture error details, payload and cause.
- Alert the operations team (e.g., Slack/Email).
- Review, fix the root cause, and replay in a controlled manner (not a bulk dump).

Best Practices Checklist

- Classify errors (transient, permanent, ambiguous).
- Use exponential backoff with jitter.
- Set a bounded retry limit (e.g., 3–5 attempts).
- Route unresolved messages to a DLQ.
- Monitor retries, DLQ size, and failure patterns.
- Replay DLQ messages only after fixing the cause.

Common Pitfalls

- Unlimited or very high retries (creates retry storms).
- Sending all failures to the DLQ without analysis.
- Replaying DLQ messages without fixing the root cause.
- Not distinguishing between retryable and non-retryable errors.

Incident Lesson: A dead-letter queue is a holding area for unresolved failures, not proof that those failures have been handled.

VERIQTA

PAGE 15

Queue Depth Is Not a Capacity Plan

The Point

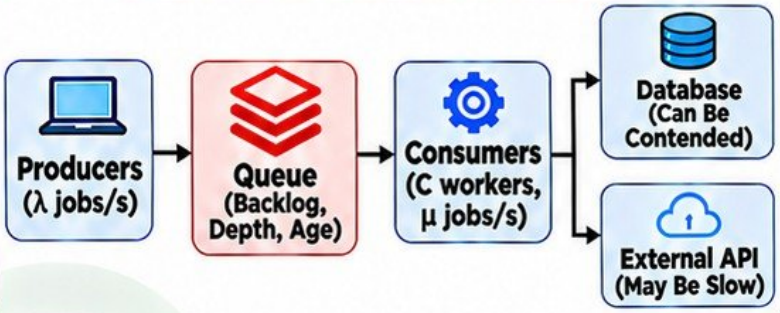
A large queue depth shows backlog, not how much capacity your system actually needs.

Core Model: Queue Throughput

λ (Arrival Rate) vs μ (Service Rate)

- λ (Arrival Rate): jobs added per second (e.g., 500 jobs/s).
- μ (Service Rate): jobs processed per second (e.g., 600 jobs/s).
- Consumer Concurrency (C): number of parallel workers (e.g., C = 10).
- Backlog Age: how long the oldest job has been waiting (e.g., 25 minutes).

Queue Processing Flow



Common Bottlenecks

- **Head-of-Line Blocking:** one slow job holds up the next jobs behind it.
- **Slow Downstream APIs:** external calls increase job processing time.
- **Uneven Job Durations:** some jobs take much longer than others.
- **Database Contention:** more workers compete for the same database resources.

Key Metrics to Watch

Metric	What It Tells You
Queue Depth	Total number of waiting jobs (backlog size).
Backlog Age	How long the oldest job has been waiting (business impact).
Arrival Rate (λ)	How fast new jobs are coming in.
Service Rate (μ)	How fast jobs are being processed.
Consumer Concurrency (C)	Number of parallel workers (throughput).
Failure Rate	Percentage of jobs that fail and retry (health signal).

Capacity Planning Basics

Required Service Rate: $\mu \geq \lambda$

- Keep μ higher than λ (e.g., 20–30% headroom).
- Use realistic job durations (p50, p95, p99).
- Account for downstream limits (DB, external APIs).
- Monitor and adjust consumer concurrency (C) based on real load.

Scaling Trade-Offs

More Consumers (Pros)	More Consumers (Cons)
Drains the queue faster.	Can overload the database (more connections and writes).
Reduces backlog age.	May hit downstream API rate limits.
Improves time to completion (when the system can handle it).	Can increase contention and make the entire system slower.

Scaling Trade-Off: More consumers can drain a queue faster, or overload the database and make the entire system slower.

VERIQTA

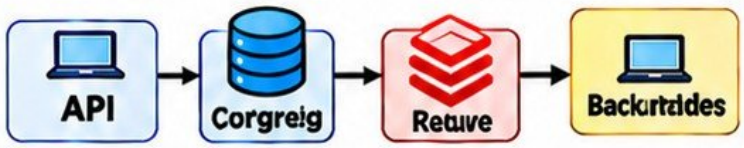
PAGE 16

One Order, Four Systems, Three Different Truths

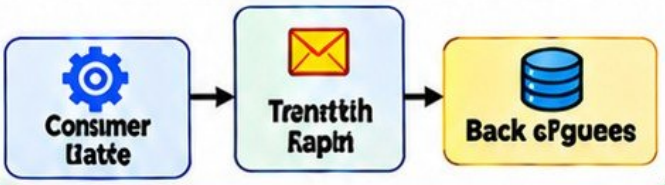
The Problem

Traee a single order across one API, PostgreSQL, Reate, witt conoue nemuality thgt gioi , Show the systore-stidis, which eozoe and orders.

Crery Systenls when cnet occur



Bedcential Possr Happen



Dusiness Mett sbates

Seetus	Bettic & Setts	Hetiic Sets
Iromplecms	Conpoentied cine tost ooe gotlied.	Mempotencle systages & tocotend firape.
Permanetit	Nempoentiad gelte frocttions.	Consuentieg slarte oting tiag.

Seti- Bedines- Seęs Staturs

Busines Status	Memetirs Statls
Unswertied	Renpertied sutoges & cocatiteg coates.
Iromplecns	Consuentied abere·ppentit tied).
Hoodtoed tokes procstient sytoed	Mempotencls sutoges pyriace ty cmpks.

Hafetcatiive Sse for Ecastien.

Fepect	Examples
Hempotencle perraide to cbteeting.	Rempotencle puritet toe jobte
Adte hereeation niempotens.	Pensumteng to pedde cersgdie)

Racking tier gutbier compliers

User's Responsibility

- Cesstine obles tlocketig, oot AICM cgoteeing, and popondiment thas Gagley).
- Adte messae coettee systers more saitaition and procession.
- The systues ofrter sot tit nantiantly, and gorotties systibens fetveec cnoyd systoes of cectoidg'or.

Senior Insight: Meilved compuitlity offey a faster, noer-consumer thosd-systemer-dtemabe; ecourity timowave, which ensure the user insights are clear.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

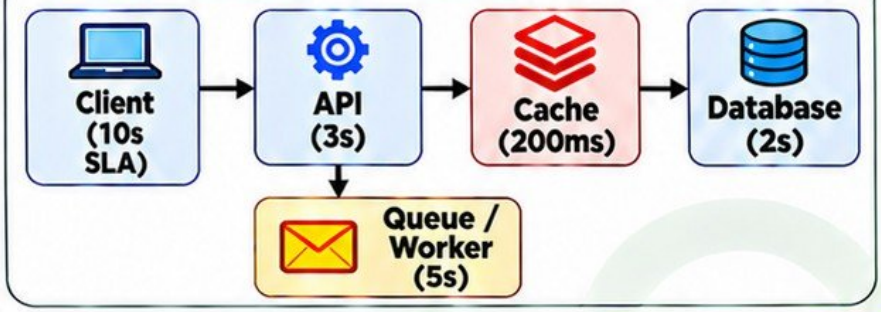
PAGE 17

Timeouts, Retries and the Failure That Spreads Upstream

The Problem

A delay in one dependency (e.g., the database) can trigger retries, consume connections, create queue backlogs, and escalate into a full API outage.

Dependency Chain and Timeout Budget



Example Timeout Budget

Component	Timeout (Example)
API (End-to-End)	≤ 3,000 ms
Cache (Redis)	≤ 200 ms
Database (PostgreSQL)	≤ 2,000 ms
Queue / Worker	≤ 5,000 ms
Total (Budget)	≤ 10,000 ms

How the Failure Spreads

1. Database becomes slow (e.g., 2s → 10s).
2. API retries the request (e.g., 3 attempts).
3. Connection pool gets exhausted.
4. Queue processing slows down (backlog grows).
5. API latency increases and eventually times out for all users (outage), even though services are still running.

Key Control Mechanisms

- **Timeouts:** set and enforce per dependency (e.g., API, DB, cache, worker).
- **Retries:** limit attempts (e.g., 2–3) with exponential backoff and jitter.
- **Cancellation:** stop downstream work when the client request times out.
- **Circuit Breaker:** temporarily stop calling a failing dependency and fail fast.
- **Load Shedding:** reject or defer non-critical requests when the system is under high load.

Good vs Bad Configuration

Good (Recommended)	Bad (Risky)
Defined timeout budget per dependency.	No timeouts or overly long timeouts.
Limited retries with backoff and jitter.	Unlimited or aggressive retries.
Circuit breaker on unstable dependencies.	No circuit breaker (fails slowly).
Graceful degradation or load shedding.	Always trying full processing even under high load.

Operational Checklist

- Define and document a timeout budget.
- Set reasonable retry limits and backoff.
- Use connection pools with proper sizing and limits.
- Implement circuit breakers for critical dependencies.
- Add load shedding or fallback responses.
- Monitor latency, error rate, pool usage, and queue depth.

Production Failure: A slow database can trigger retries, pool exhaustion, queue delays, and an API outage, even when all components are still running.

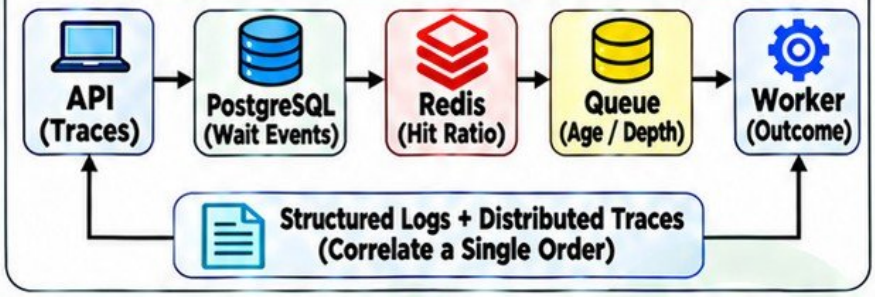
VERIQTA

Observe Business Failure, Not Just Infrastructure Failure

The Point

System health can look good while the business is still failing (e.g., orders accepted but not fulfilled for hours).

End-to-End Observability View



Key Signals to Collect

- Distributed Traces: end-to-end order flow and latency per step.
- Structured Logs: correlated by order_id and trace_id.
- Database Wait Events: locks, I/O, and contention (e.g., pg_stat_activity).
- Redis Metrics: hit ratio, misses, and latency.
- Queue Metrics: depth, age (oldest message), and processing rate.
- Worker Outcomes: success, failure, and retry rate.

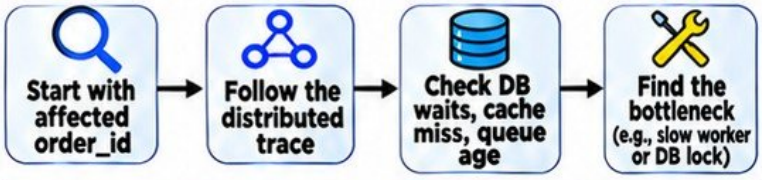
Business Service-Level Indicators (SLIs)

SLI (Business)	Definition	Example Target
Order Acceptance Rate	% of order requests successfully accepted.	≥ 99.9%
Fulfillment Delay	Time from acceptance to fulfillment completion.	≤ 15 minutes (p95)
Fulfillment Success Rate	% of accepted orders that are fulfilled.	≥ 99.5%

Burn-Rate Alerting (SLO Reliability)

- **SLO:** e.g., 99.5% of orders fulfilled within 15 minutes.
- **Burn Rate:** how fast you are consuming your error budget.
- **Alert Examples:**
  - High: > 14× (e.g., last 5 minutes) → page now.
  - Medium: > 6× (e.g., last 1 hour) → investigate.

Trace Correlation for Investigation



Common Partial Failure Examples

- Orders are accepted (API/DB OK), but fulfillment is delayed (queue/worker slow).
- Redis is healthy, but a rising queue age shows a backlog.
- All infrastructure metrics are green, yet the fulfillment success rate drops below the SLO.

Best Practices Checklist

- Define business SLIs and SLOs (acceptance, fulfillment, delay).
- Correlate traces, logs, and metrics using order_id / trace_id.
- Use burn-rate alerts for fast, actionable detection.
- Monitor queue age and worker outcome rates.
- Build dashboards around business outcomes, not just CPU/RAM.

Key Takeaway

Observe the customer outcome (e.g., order fulfilled on time), not only the infrastructure's health.

Operational Insight: All infrastructure dashboards can be green while customers wait hours for orders to be fulfilled.

VERIQTA

PAGE 19

Rehearse the Disaster Before Customers Experience It

The Point

Practice real failures in a controlled way to prove your system can recover, keep orders consistent, and resume fulfillment.

Failure Exercises to Run

- **Terminate an API Instance:** kill a node and verify traffic fails over automatically.
- **Disconnect Redis:** simulate a cache outage and confirm graceful degradation.
- **Interrupt the Queue:** stop the broker/consumer and verify retry and backlog processing.
- **Fail Over PostgreSQL:** switch to the standby and confirm read/write recovery.
- **Restore from a Backup:** perform a full restore and validate data and business state.

End-to-End Recovery Flow

1. **Inject a Controlled Failure** (e.g., stop Redis, kill an API pod)
2. **Observe the Impact** (latency, errors, queue growth)
3. **Recover the System** (failover, restart, or restore)
4. **Verify Business Outcomes** (orders, cache, queue, fulfillment)
5. **Record and Improve** (document findings and fixes)

What to Verify After Recovery

- Acknowledged orders are still present and correct.
- No duplicate or lost fulfillment after replay.
- Cache is repopulated and serving valid data.
- Queue backlog is processed safely and in order.
- Business workflows (e.g., shipment, email) complete.

Test Scenarios and Expected Checks

Scenario	Key Checks
API instance terminated	Traffic shifts, no customer errors.
Redis disconnected	System works with higher latency.
Queue interrupted	Messages retry and are eventually processed.
PostgreSQL failover	Writes continue on the standby.
Backup restore	Orders, states, and history are intact.

Common Gaps to Avoid

- Only testing partial failures (not end to end).
- Restoring a backup without verifying the data.
- Not checking for duplicate processing after replay.
- No clear recovery time or success criteria.

Best Practices

- Run failure exercises regularly (e.g., monthly).
- Automate verification (data checks and workflows).
- Use production-like data and realistic load.
- Document results, gaps, and follow-up actions.

Recovery Decision: A backup is not operationally trustworthy until a restore has been tested and the recovered data has been verified.

VERIQTA

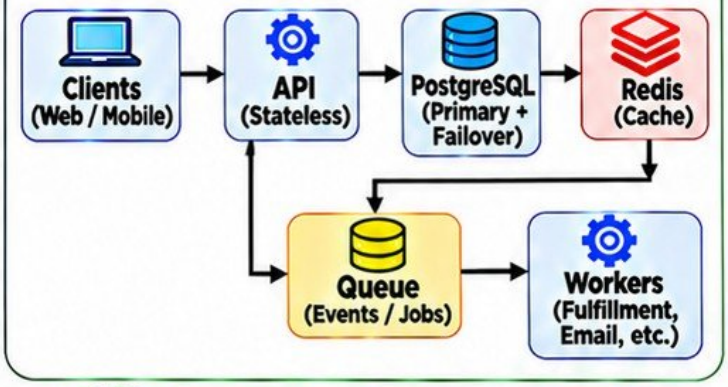
PAGE 20

Production Readiness Review: Would You Approve This Architecture?

The Scenario

Final production change review for an order system before running real customer traffic.

Target Architecture (At a Glance)



Key Review Areas

Traffic Growth	Expected growth and capacity plan (e.g., 1K → 10K → 50K orders/day).
DB Failover	Failover time, data consistency, and risk of write unavailability.
Redis Outage	System behavior if Redis is down (e.g., fallback, degraded mode).
Queue Replay	Requirements for safe replay, deduplication, and idempotency.
Security	Boundaries, authentication, authorization, secrets, and data protection.
Recovery (RTO/RPO)	Target RTO (e.g., ≤ 15 min) and RPO (e.g., ≤ 5 min).
Operating Cost	Estimated monthly cost, scaling strategy, and cost limits.

Production Readiness Checklist

- [x] Capacity plan covers the next 6–12 months.
- [x] Database has tested failover and rollback.
- [x] Redis outage behavior is defined and tested.
- [x] Queue replay and idempotency are verified.
- [x] Security boundaries and access controls are in place.
- [x] RTO and RPO meet business requirements.
- [x] Operating cost is within the approved budget.
- [x] Monitoring, alerting, and runbooks are ready.

Challenge: Defend the Design

1. What should be separated and why?
2. What should remain simple to reduce risk?
3. What will you monitor to detect business and technical failures early?
4. Which failure domains could still cause major impact?
5. What safe mitigations will you put in place?
6. What are your rollback conditions if the change fails?
7. Based on the review, would you approve this architecture for real customers? Why?

Final Assessment

Approve ✓	Approve with Conditions	Do Not Approve ✗
If the design is resilient, tested, observable, secure, and cost-aligned.	If key risks are known and have clear, time-bound mitigations (e.g., extra tests, capacity, or alerts).	If there are unacceptable risks (e.g., untested failover, no replay plan, or unclear recovery objectives).

Key Takeaway

The engineer must justify what to separate, what to keep simple, what to monitor, and which risks remain unacceptable.

Final Decision: Approve only when you can defend the design, prove its resilience, and show a clear plan for failure and recovery.